

Probeklausur 2 – Digitales Bibliotheks- und Medienverwaltungssystem

Bearbeitungszeit: 2,5 h**Typ:** Einzelarbeit**Erreichbare Punkte:** 100

Hinweis: Diese Probeklausur dient der eigenständigen Prüfungsvorbereitung und orientiert sich am Aufbau der Modulprüfung. Lesen Sie zunächst die gesamte Problemstellung, bevor Sie mit der Bearbeitung beginnen.

1. Problemstellung

Eine moderne Stadtbibliothek möchte ihren digitalen Medienbestand objektorientiert verwalten. Dafür soll ein wiederverwendbares und erweiterbares Softwaresystem in C++ entwickelt werden.

Der digitale Medienbestand besteht aus unterschiedlichen Medientypen:

- E-Book
- Hörbuch

Alle Medien besitzen gemeinsame Eigenschaften:

- eine eindeutige Medien-ID
- den Titel
- den Verfügbarkeitsstatus (verfügbar oder ausgeliehen)
- die erforderliche Zugriffsberechtigung (z. B. „Standard“, „Fachbereich“, „Premium“)
- das momentan ausleihende Mitglied

Ein E-Book speichert zusätzlich noch seine Dateigröße in MB. Bei Hörbüchern interessiert insbesondere die Laufzeit in Minuten. Neu erstellte Medien sollen zu Beginn verfügbar sein und kein ausleihendes Mitglied besitzen.

Zu jedem Objekt eines Medientyps lassen sich mit der Funktion **printInfo()** alle Attribute in einer Tabelle mit zwei senkrecht ausgerichteten Spalten auf dem Terminal ausgeben.

Zwei Ausgabebeispiele für ein Objekt vom Typ „E-Book“ und „Hörbuch“ sehen wie folgt aus:

Type	E-Book
Title	Clean Code
File Size	12.5 MB
Available	Yes
Required Access	Fachbereich
Borrowed By	None

Type	Audiobook
Title	Der Hobbit
Duration	660 min
Available	No
Required Access	Standard
Borrowed By	Anna Mueller

Auch die Daten der angemeldeten Mitglieder werden über die Software verwaltet. Ein Mitglied besitzt eine ID, einen Namen sowie eine Liste seiner freigeschalteten Zugriffsberechtigungen (z. B. „Standard“, „Fachbereich“, „Premium“). Die ID soll nur einmalig gesetzt werden können und muss eindeutig über alle Mitglieder hinweg sein. Da sich die freigeschalteten Berechtigungen ändern können, soll ein passender Container aus der Standard-Bibliothek verwendet werden, der geordnet bzw. effizient durchsucht werden kann. Es sollen Methoden vorhanden sein, um einem Mitglied eine neue Berechtigung hinzuzufügen oder eine bestehende Berechtigung zu entfernen.

Mitglieder können maximal ein Medium gleichzeitig ausleihen, aber nur wenn dieses Medium verfügbar ist, das Mitglied aktuell kein anderes Medium ausgeliehen hat und es die erforderliche Zugriffsberechtigung des Mediums besitzt.

Damit nicht für jeden zu verwaltenden Typ (Medien, Mitglieder, ...) eine eigene Verwaltungsklasse geschrieben werden muss, soll eine **generische, wiederverwendbare Verwaltungsklasse mithilfe eines Templates** implementiert werden. Diese Verwaltungsklasse speichert ihre Elemente in einem Vector-Container und bietet mindestens folgende Methoden:

- Hinzufügen eines neuen Elements
- Ermitteln der Anzahl gespeicherter Elemente
- Suchen und Zurückgeben eines Elements anhand seiner ID

Die verwalteten Objekte sollen über **Smart Pointer** gehalten werden, sodass keine manuelle Speicherfreigabe (delete) im Anwendungscode notwendig ist.

Weitere Requirements:

- Verwenden Sie mindestens eine **abstrakte Klasse** und ein **statisches Attribut**.
- Verwenden Sie mindestens eine **Template-Klasse oder Template-Funktion**.
- Verwalten Sie dynamisch erzeugte Objekte ausschließlich über **Smart Pointer** (kein nacktes new/delete im Anwendungscode).
- Orientieren Sie sich am Prinzip der **Datenkapselung** und implementieren Sie die nötigen Getter- & Setter-Methoden.
- Achten Sie auf **const-Korrektheit**, wo dies sinnvoll ist.
- Stellen Sie sicher, dass Ihr Code **wiederverwendbar und erweiterbar** ist.

2. Abgabebumfang

Folgende Aufgaben müssen bearbeitet werden:

a) UML-Klassendiagramm (20 Punkte)

Erstellen Sie ein UML-Klassendiagramm, das als Basis für das geplante Softwaresystem dient und die oben beschriebenen Anforderungen erfüllt. Darzustellen sind:

- Alle Attribute und Methoden
- Sichtbarkeiten
- Datentypen
- Zusätzliche Eigenschaften bei konstanten, statischen, abgeleiteten, eindeutigen, virtuellen, abstrakten usw. Methoden/Attributen/Parametern
- Parameterrichtungen
- Klassenbeziehungen mit Navigierbarkeit, Multiplizität und Beziehungsnamen

- Die generische Verwaltungsklasse in korrekter UML-Notation für **Templates** (parametrisierte Klasse).

Hinweis: Um Zeit bei Ihrem Diagramm zu sparen, genügt es pro Klasse jeweils nur eine Getter- und eine Setter-Methode darzustellen.

b) Implementierung in C++ (45 Punkte)

Implementieren Sie nun das Softwaresystem zur Verwaltung des oben beschriebenen Medienbestands in C++. Folgen Sie dabei der Coding Convention für C++ aus der Vorlesung.

Wichtig: Orientieren Sie sich an Ihrem zuvor erstellten Klassendiagramm. D. h. der Code sollte alle Spezifikationen Ihres Klassendiagramms erfüllen. Erweitern Sie den Code auch um alle technischen Details, die typischerweise nicht in Klassendiagrammen abgebildet werden, aber aus der Aufgabenbeschreibung hervorgehen und für die korrekte Umsetzung der Software notwendig sind (z. B. Konstruktoren, virtueller Destruktor).

Hinweis 1: Hierbei müssen nun alle relevanten Getter- & Setter-Methoden implementiert werden.

Hinweis 2: Sie müssen KEINE Funktions-Header, wie in den Coding Conventions gefordert, dokumentieren.

Hinweis 3: Die Ausleih-Logik muss alle genannten Bedingungen prüfen (Medium verfügbar, Mitglied hat noch kein Medium ausgeliehen, passende Zugriffsberechtigung vorhanden) und bei einer Verletzung eine geeignete Rückmeldung geben (z. B. Rückgabewert `bool` oder eine Exception).

c) main-Funktion (15 Punkte)

- Erzeugen Sie für jeden Medientyp mindestens ein Objekt sowie mindestens ein Mitglied.
- Speichern Sie alle Medien über Smart Pointer in einem generischen Verwaltungsobjekt (Vector-Container). Speichern Sie alle Mitglieder ebenfalls in einem Verwaltungsobjekt.
- Führen Sie mindestens einen Ausleihvorgang durch – darunter auch einen Fall, der fehlschlägt, weil eine der Bedingungen verletzt ist.
- Geben Sie zum Schluss in einer Schleife die Daten aller Medien über einen **Basisklassenzeiger** aus (`printlnf()`).

d) Kompilierbarkeit (5 Punkte)

Das Programm muss erfolgreich/fehlerfrei auf Ihrer Prüfungsmaschine kompilierbar und lauffähig sein.

e) Wissensfragen (15 Punkte)

Beantworten Sie die folgenden Fragen direkt als Kommentar in Ihrem Code:

- 1 Wie können Sie bei polymorphen Klassen sicherstellen, dass die korrekte virtuelle Funktion überschrieben wird?
- 2 Erklären Sie den Unterschied zwischen `std::unique_ptr` und `std::shared_ptr`. Welchen würden Sie zum Speichern der Medien im Verwaltungs-Vector wählen und warum?
- 3 Was ist eine abstrakte Klasse, wie deklarieren Sie in C++ eine rein virtuelle Methode, und warum lässt sich von einer abstrakten Klasse kein Objekt instanziiieren?
- 4 Warum sollte der Destruktor Ihrer Basisklasse virtuell sein? Beschreiben Sie, was ohne virtuellen Destruktor passieren kann.
- 5 Welchen Vorteil bietet der Einsatz von Templates? Nennen Sie ein konkretes Beispiel aus Ihrer Implementierung.
- 6 Was versteht man unter dem RAII-Prinzip und wie unterstützen Smart Pointer dieses Prinzip?

Bewertungsübersicht

Aufgabe	Inhalt	Punkte
a)	UML-Klassendiagramm	20
b)	Implementierung in C++	45
c)	main-Funktion	15
d)	Kompilierbarkeit / Lauffähigkeit	5
e)	Wissensfragen	15
	Gesamt	100